

Lecture 3

The Application Layer

Part 1

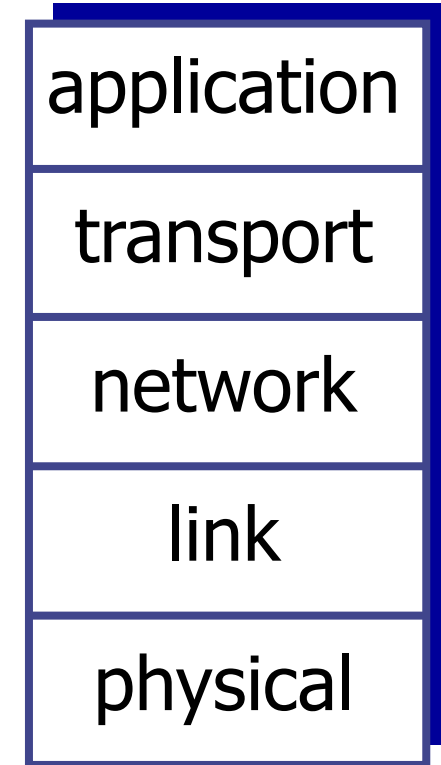
Subjects of today:

- The Role of the Application Layer
- Apps and protocols
- Architectures
- Interfacing the Application Layer
- Web and HTTP

3.1 The Role of the Application Layer

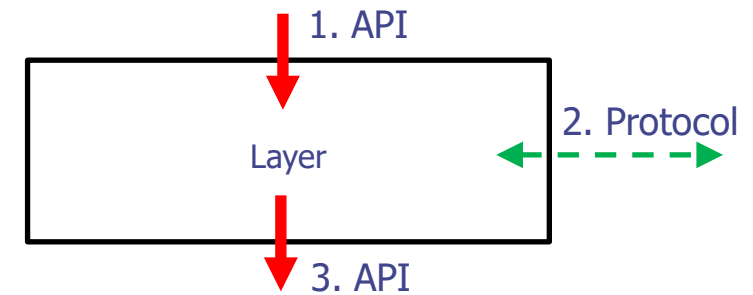
The Internet Protocol Stack (again...)

- ***application***: supporting network applications
- *transport*: process-process data transfer
- *network*: routing of datagrams from source to destination
- *link*: data transfer between neighboring network elements
- *physical*: bits “on the wire”



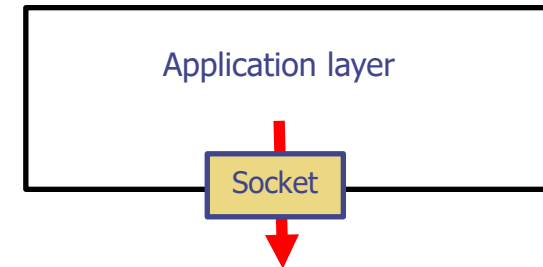
Every layer must...

1. offer services to an upper layer.
2. comply with agreed protocols.
3. utilize services from the underlying layer.



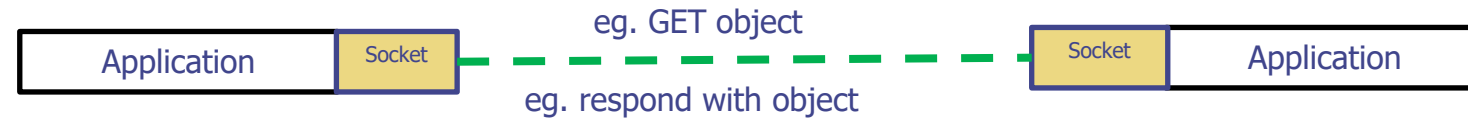
The application layer...

- has no upper layer.
- hosts the applications.
- utilizes services from the transport layer through sockets.

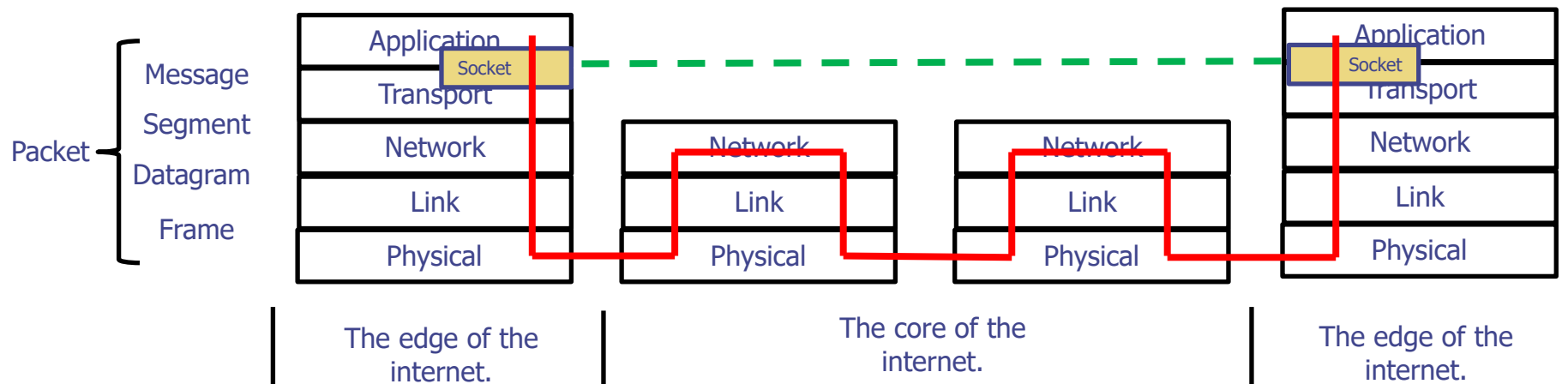


Scoping

Application view



Real view



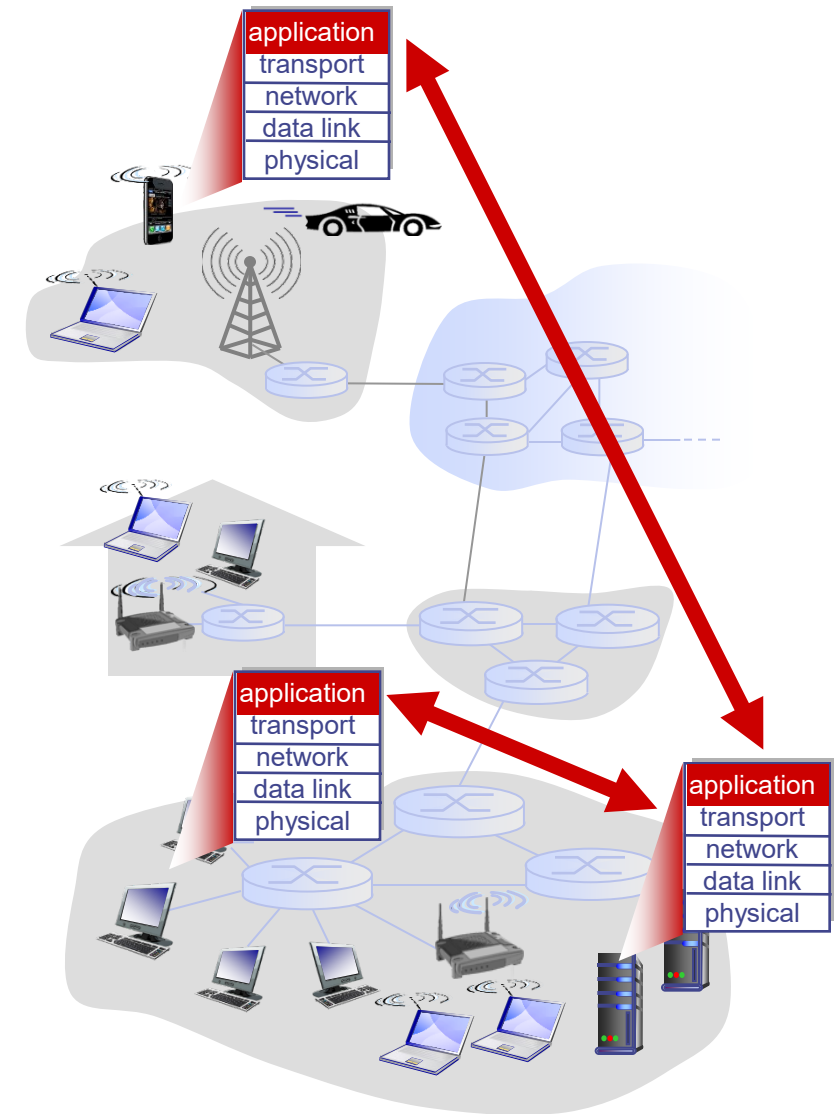
Creating a network app

Write programs that:

- run on (different) *end systems*
- communicate over network
- e.g., web server software communicates with browser software

No need to write software for network-core devices as:

- network-core devices do not run user applications
- maintaining applications only on end systems allows for rapid app development



3.2 Apps and Protocols

Example network apps

- Web browsing
- E-mail
- remote login
- P2P file sharing
- Text messaging
- Voice over IP (VoIP)
- Real-time video conferencing
- Streaming stored video
- Multi-user network games
- Social networking
- Search
- IoT Device Communication
- Etc.

Some application layer protocols

Application	Protocol examples
Mail	SMTP (sending) POP3 (receiving) IMAP (Syncing)
Web	HTTP
Login	TELNET SSH
File sharing	FTP SFTP
Proprietary application	Standard protocols
Proprietary application	Proprietary protocols

3.3 Architectures

Application architectures

Possible structure of applications:

- client-server
- peer-to-peer (P2P)
- Publisher-subscriber (especially at IOT)
- The cloud

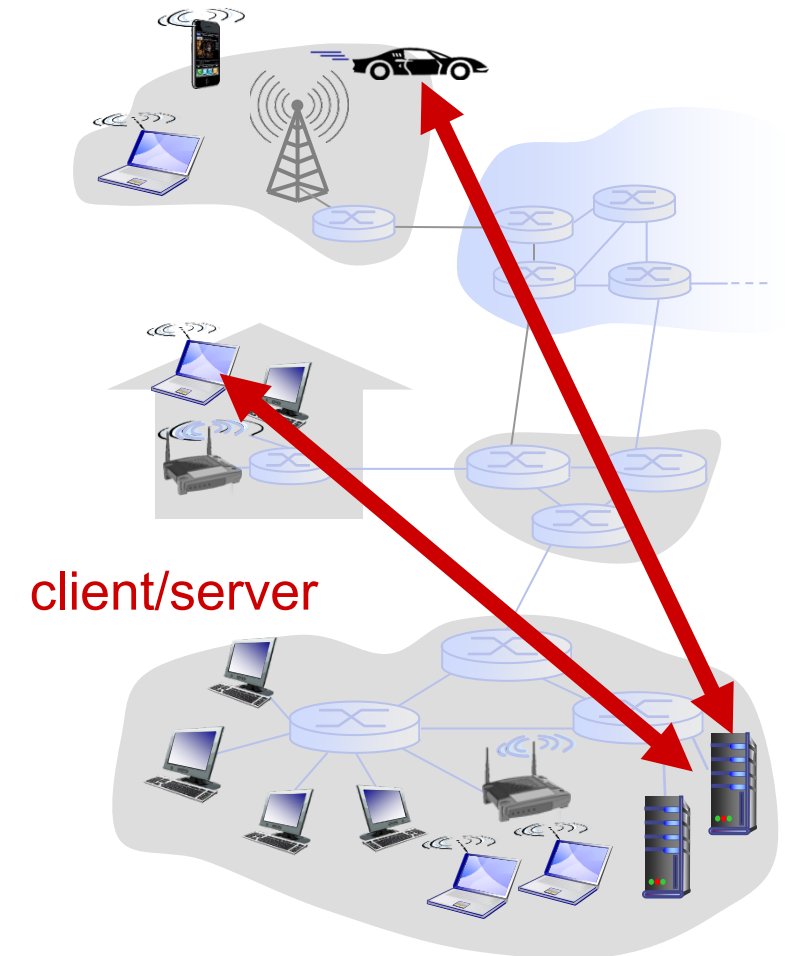
Client-server architecture

Server:

- Always-on host
- Permanent IP address
- Data centers for scaling

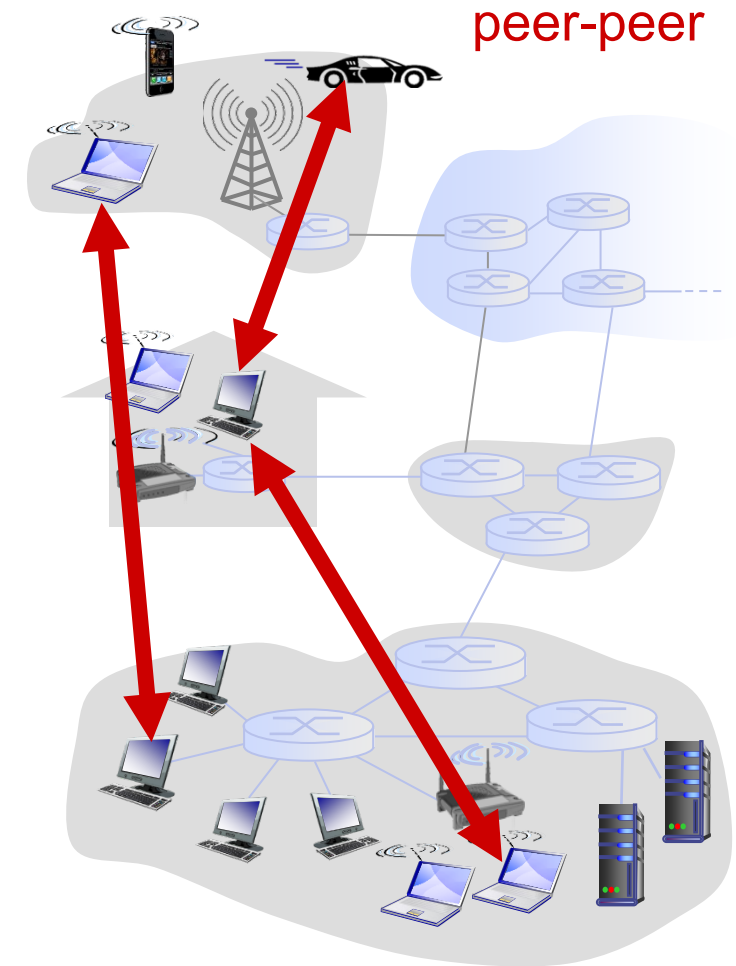
Clients:

- Communicate with server
- May be intermittently connected
- May have dynamic IP addresses
- Do not communicate directly with each other



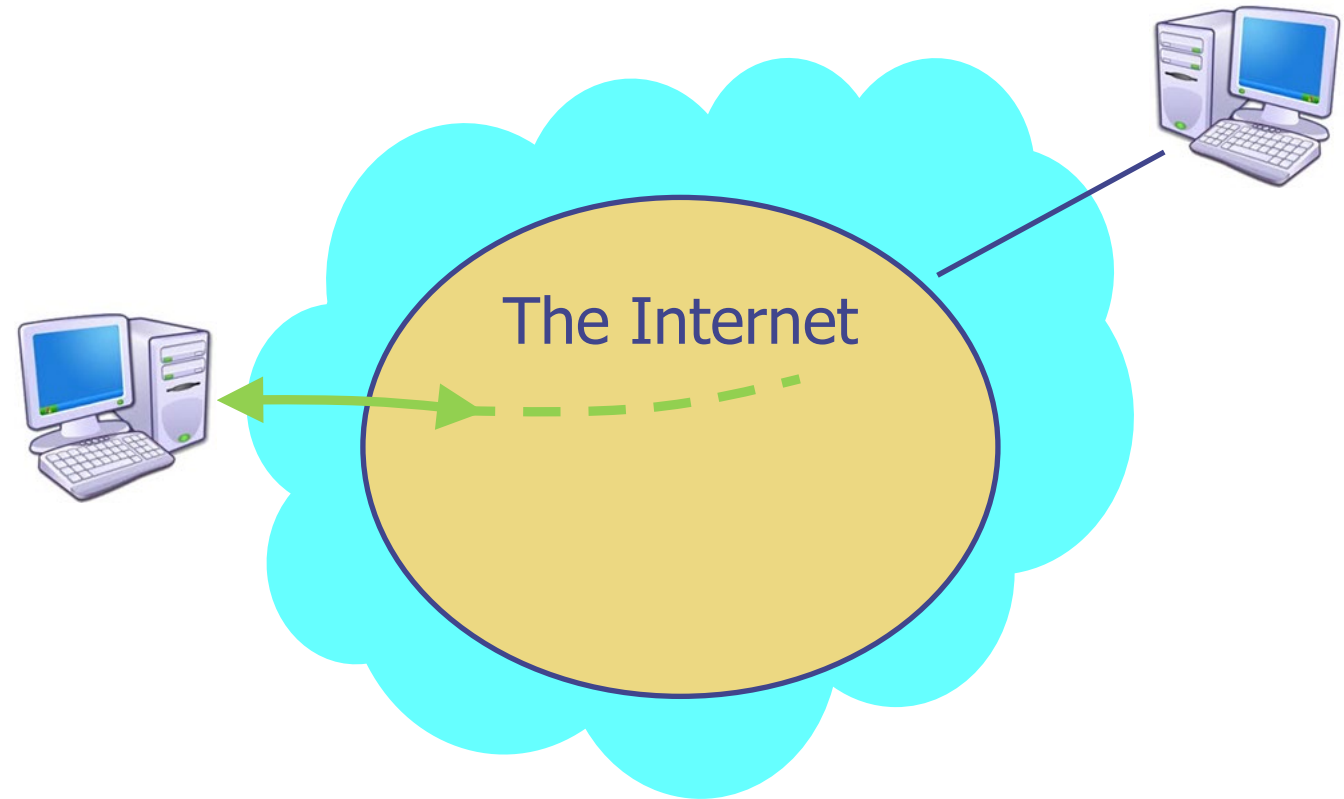
P2P architecture

- *Not* always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
 - ***Self scalability*** – new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
 - Complex management



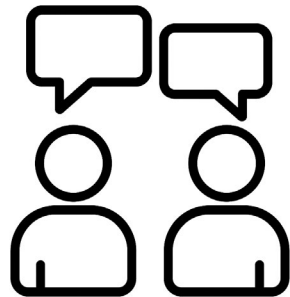
Cloud architecture

- Decentralized “virtual” server
 - Dynamic resource management
 - Scalable
 - Complex

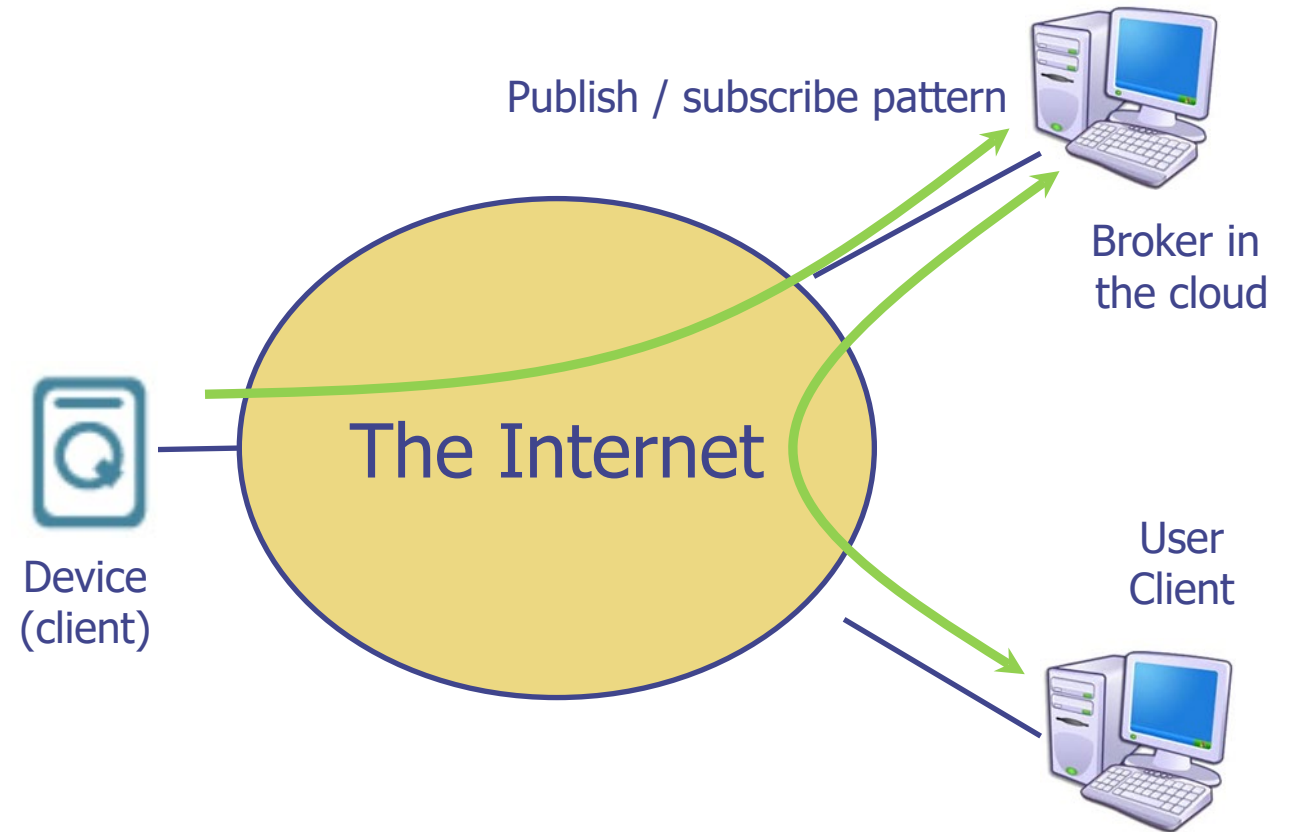


Publisher-Subscriber architecture

- Register *Topic* at *Message Broker*
- Example: MQTT (see <https://mqtt.org/>)
 - Lightweight and Efficient
 - Scalable
 - Bi-directional Communications



Which applications
could be relevant for
this? (think relative to
your field of study)



3.4 Interfacing the Application Layer

Processes communicating

Process: program running within a host

- Within same host, two processes communicate using **inter-process communication** (defined by OS)
- Processes in different hosts communicate by exchanging **messages**

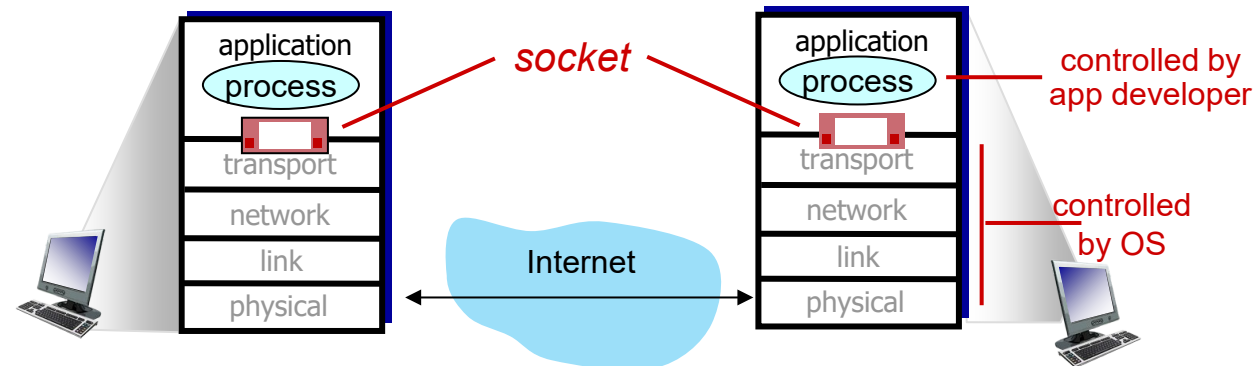
Definition

client process: process that initiates communication

server process: process that waits to be contacted

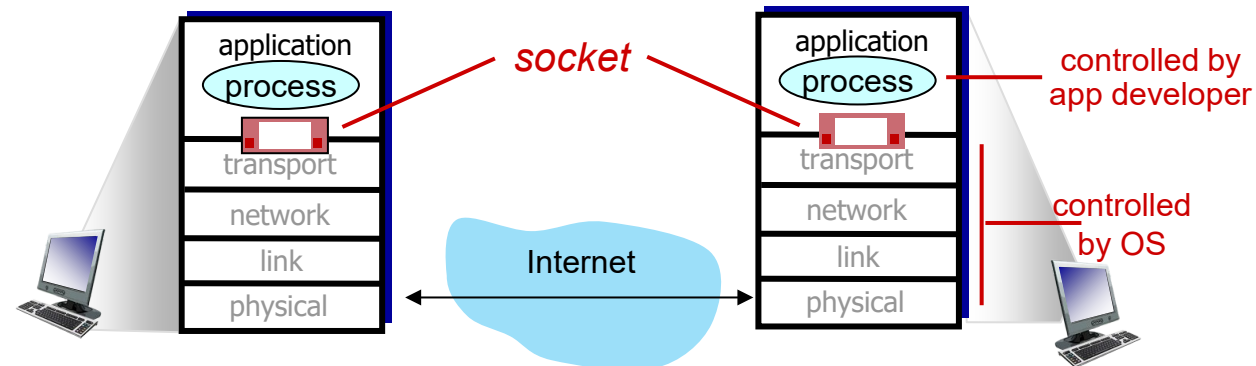
Sockets

- Process sends/receives messages to/from its socket
- Socket analogous to door
- Two sockets involved: one on each side



Sockets

- Sending process shoves message out of the door
- Sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process



- Receiving process gets message from the door
- Receiving process relies on transport infrastructure on other side of door to deliver message from socket at sending process

Addressing Processes

- To receive messages, a process must have *identifier*
- Host device has unique 32-bit IP address
- Q: Does the IP address of a host on which process runs suffice for identifying the process?
- A: no, *many* processes can be running on same host
- *Identifier* includes both **IP address** and **port numbers** associated with process on host.
- Example port numbers:
 - HTTP server: 80
 - mail server: 25
- To send HTTP message to gaia.cs.umass.edu web server:
 - **IP address:** 128.119.245.12
 - **port number:** 80

App-layer Protocol Defines

- Types of messages exchanged
 - e.g., request, response
 - Message syntax:
 - what fields in messages & how fields are delineated
 - Message semantics
 - meaning of information in fields
 - Rules for when and how processes send & respond to messages
-
- Open protocols defined in RFCs and allows for interoperability

What transport service does an app need?

Reliable data transfer

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

Timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

Throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

Security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video: 10kbps-5Mbps	yes, 100's ms
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's ms
text messaging	no loss	elastic	Depends

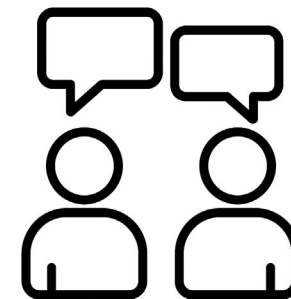
Internet Transport Protocols Services

TCP service:

- **Reliable transport** between sending and receiving process
- **Flow control:** sender won't overwhelm receiver
- **Congestion control:** throttle sender when network overloaded
- **Does not provide:** timing, minimum throughput guarantee, security
- **Connection-oriented:** setup required between client and server processes

UDP service:

- **Unreliable data transfer** between sending and receiving process
- **Does not provide:** reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,



Why bother? Why is there a UDP?

Applications and their transport protocols

<u>application</u>	<u>application layer protocol</u>	<u>underlying transport protocol</u>
e-mail	SMTP [RFC 2821]	TCP
remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
file transfer	FTP [RFC 959]	TCP
streaming multimedia	HTTP, RTP [RFC 1889]	TCP or UDP
Internet telephony	SIP, RTP, proprietary	TCP or UDP

Securing TCP

TCP & UDP

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext

Transport Layer Security (TLS)

- provides encrypted TCP connection
- data integrity
- end-point authentication

TLS implemented at App. layer

- Apps use TLS libraries, that use TCP in turn

TLS socket API

- Cleartext passwords sent into socket traverse Internet encrypted
- *See Chapter 8*

3.5 Web and HTTP

Web and HTTP

- *A web page* consists of *objects*
 - The object can be HTML file, JPEG image, Java applet, audio file,...
 - A web page consists of *base HTML-file* which includes *several referenced objects*
-
- Each object is addressable by a *URL*, e.g.:

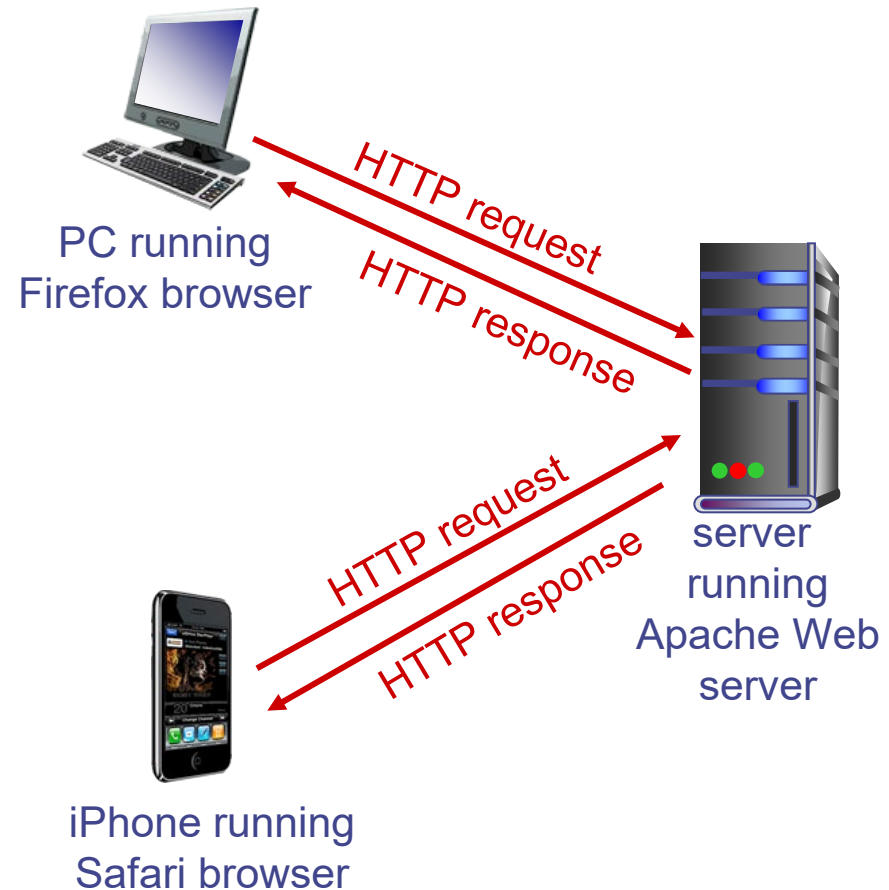
`www.someschool.edu/someDept/pic.gif`

host name

path name

HTTP Overview

- **H**yper**T**ext **T**ransfer **P**rotocol
 - Text with hyperlinks
 - Web's application layer protocol
- client/server model
 - *client*: browser that requests, receives and displays Web objects
 - *server*: Web server sends objects in response to requests
 - Both use the HTTP protocol

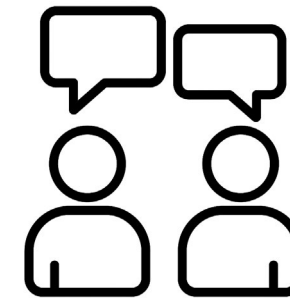


HTTP overview (continued)

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed
- HTTP is *stateless*
 - server maintains no information about past client requests
 - A state for http can be achieved through *cookies*

Maintain a state is complex!

- Past history (state) must be maintained
- If server/client crashes, their views of “state” may be inconsistent, must be reconciled



What could a
state be for
http?

HTTP connections

Non-persistent HTTP

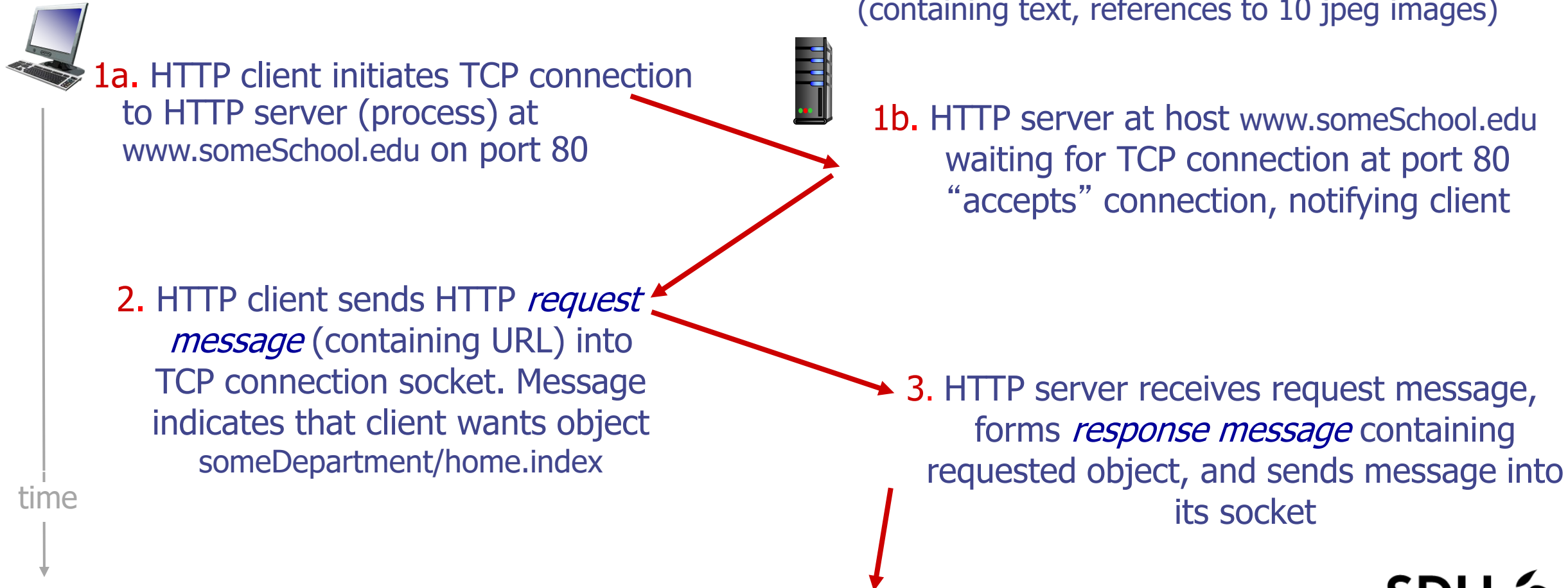
- Max one object sent over the (TCP) connection
 - connection then closed

Persistent HTTP

- Multiple objects can be sent over a single (TCP) connection between client and server

Non-persistent HTTP

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



Non-persistent HTTP (cont.)



5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

4. HTTP server "initiates" closing of the TCP connection.

6. Steps 1-5 repeated for each of 10 jpeg objects

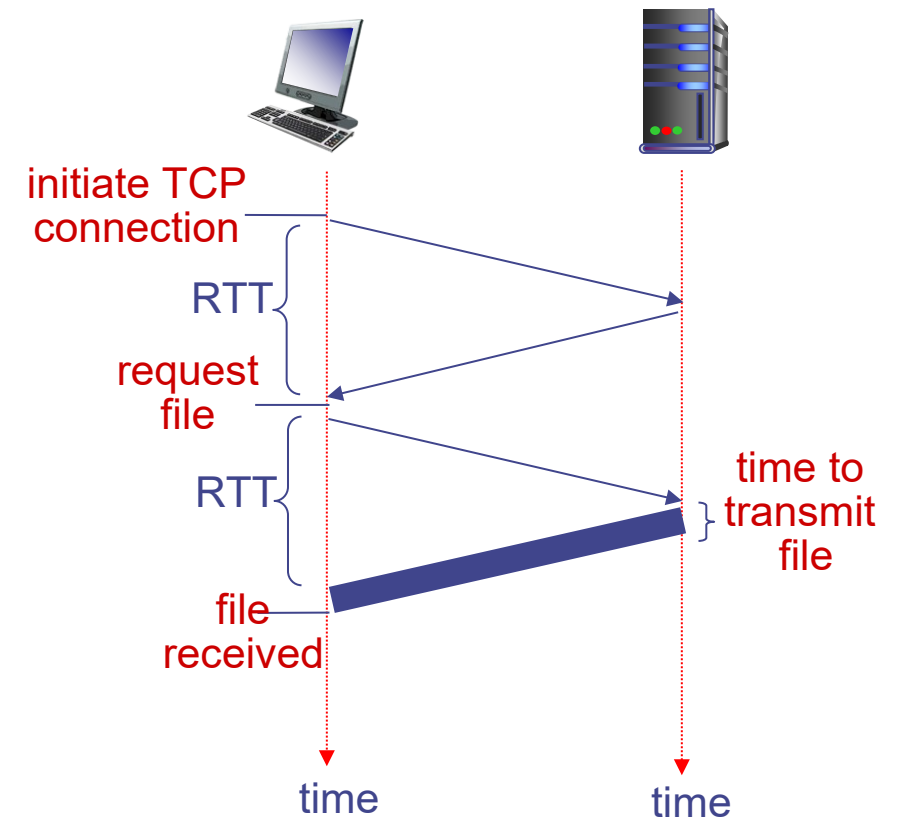
time

Non-persistent HTTP: response time

RTT (definition): time for a small (single) packet to travel from client to server and back

HTTP response time:

- One RTT to initiate TCP connection
- One RTT for HTTP request and first few bytes of HTTP response to return
- File transmission time
- Response time = $2RTT + \text{file transmission time}$



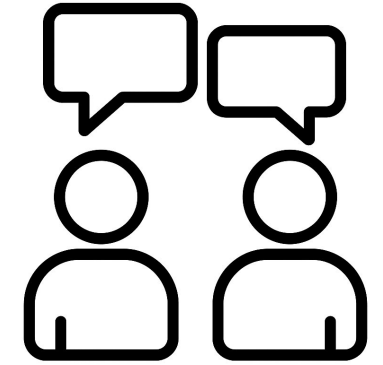
Persistent HTTP

Non-persistent HTTP issues:

- Requires 2 RTTs per object
- OS overhead for **each** TCP connection
- Browsers often open parallel TCP connections to fetch referenced objects

Persistent HTTP:

- Server leaves connection open after sending response
- Subsequent HTTP messages between same client/server sent over open connection
- Client sends requests as soon as it encounters a referenced object
- Just one RTT for all the referenced objects

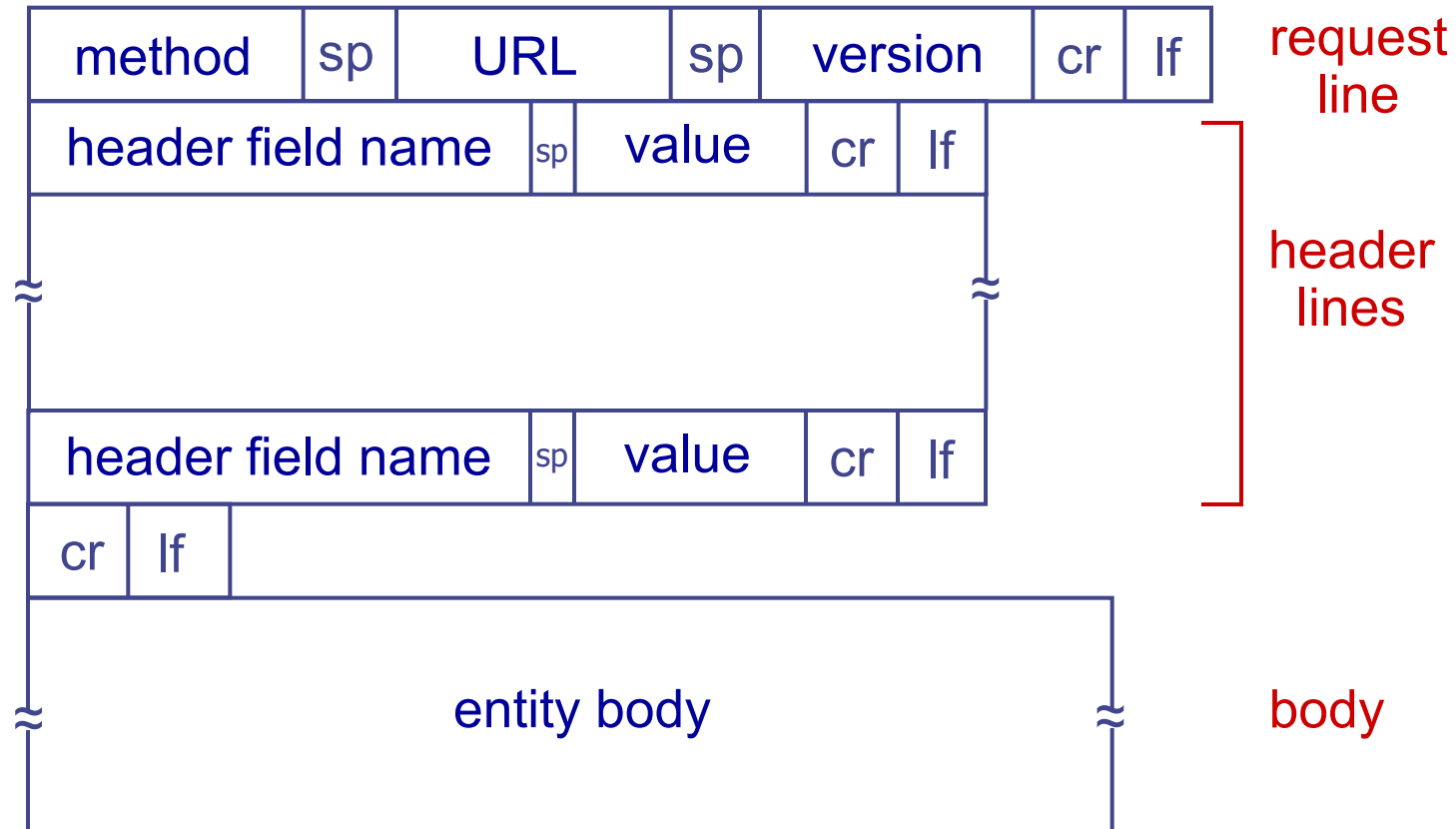


Why can
multiple parallel
connections be
problematic?

App-layer Protocol Defines (again)

- Types of messages exchanged
 - e.g., request, response
- Message syntax
- Message semantics
- Rules for when and how processes send & respond to messages
 - Request → response
 - Request → response
 - ...
 - Request → response
 - Done.

HTTP request message: general format



HTTP request message

- Two types of HTTP messages: *request, response*
- HTTP request message: ASCII (human-readable format)

request line
(GET, POST,
HEAD commands)

header
lines

carriage return,
line feed at start
of line indicates
end of header lines

carriage return character
line-feed character

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

HTTP GET seen in wireshark.

The image shows a Wireshark network traffic capture. The top pane displays a list of captured packets, with packet 4 selected. The middle pane shows the details of the selected packet, highlighting the Hypertext Transfer Protocol (HTTP) section. The bottom pane shows the raw packet data in hexadecimal and ASCII.

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
4	11:22:13,803915	10.94.58.75	128.119.245.12	HTTP	540	GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
6	11:22:13,916472	128.119.245.12	10.94.58.75	HTTP	492	HTTP/1.1 200 OK (text/html)
7	11:22:13,955631	10.94.58.75	128.119.245.12	HTTP	412	GET /favicon.ico HTTP/1.1

Packet Details:

- Frame 4: 540 bytes on wire (4320 bits), 540 bytes captured (4320 bits) on interface
- Ethernet II, Src: IntelCor_97:11:bb (58:a0:23:97:11:bb), Dst: All-HSRP-router
- Internet Protocol Version 4, Src: 10.94.58.75, Dst: 128.119.245.12
- Transmission Control Protocol, Src Port: 62322, Dst Port: 80, Seq: 1, Ack: 1
- Hypertext Transfer Protocol**
 - GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
 - Host: gaia.cs.umass.edu\r\n
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/115.0\r\n
 - Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8\r\n
 - Accept-Language: da,en-US;q=0.7,en;q=0.3\r\n
 - Accept-Encoding: gzip, deflate\r\n
 - Connection: keep-alive\r\n
 - Upgrade-Insecure-Requests: 1\r\n
 - If-Modified-Since: Tue, 12 Sep 2023 05:59:01 GMT\r\n
 - If-None-Match: "51-6052323b6f938"\r\n
 - \r\n
 - [Full request URI: <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>]
 - [HTTP request 1/2]
 - [Response in frame: 6]
 - [Next request in frame: 7]
 - [Community ID: 1:4VtvSTsp8C+LRmCrGK+VDM0FLZ4=]
- TRANSMISSION DATA

Raw Data:

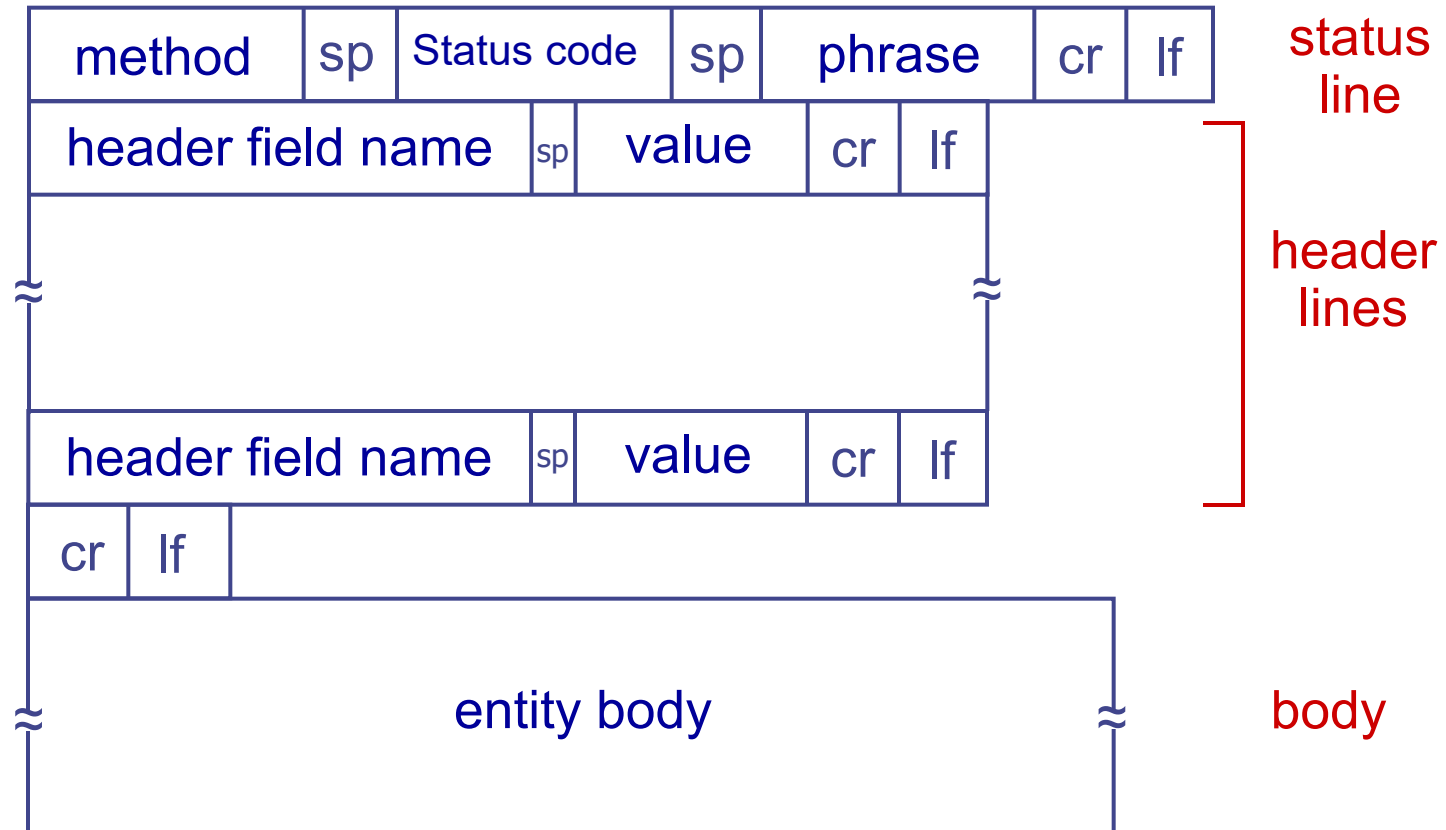
```

0000  00 00 0c 07 ac 01 58 a0 23 97 11 bb 08 00 45 00  .....X. #.....E.
0010  02 0e 38 34 40 00 80 06 06 89 0a 5e 3a 4b 80 77  ..84@... ..^:K.w
0020  f5 0c f3 72 00 50 6b bc 4e 64 d0 a0 de 19 50 18  ...r.Pk. Nd....P.
0030  02 00 05 2a 00 00 47 45 54 20 2f 77 69 72 65 73  ...*.GE T/wires
0040  68 61 72 6b 2d 6c 61 62 73 2f 49 4e 54 52 4f 2d  hark-lab s/INTRO-
0050  77 69 72 65 73 68 61 72 6b 2d 66 69 6c 65 31 2e  wireshar k-file1.
0060  68 74 6d 6c 20 48 54 54 50 2f 31 2e 31 0d 0a 48  html HTT P/1.1..H
0070  6f 73 74 3a 20 67 61 69 61 2e 63 73 2e 75 6d 61  ost: gai a.cs.uma
0080  73 73 2e 65 64 75 0d 0a 55 73 65 72 2d 41 67 65  ss.edu... User-Age
0090  6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30 20  nt: Mozi lla/5.0
00a0  28 57 69 6e 64 6f 77 73 20 4e 54 20 31 30 2e 30  (Windows NT 10.0
00b0  3b 20 57 69 6e 36 34 3b 20 78 36 34 3b 20 72 76  ; Win64; x64; rv
00c0  3a 31 30 39 2e 30 29 20 47 65 63 6b 6f 2f 32 30  :109.0) Gecko/20
00d0  31 30 30 31 30 31 20 46 69 72 65 66 6f 78 2f 31  100101 F irefox/1
00e0  31 35 2e 30 0d 0a 41 63 63 65 70 74 3a 20 74 65  15.0..Ac cept: te
00f0  78 74 2f 68 74 6d 6c 2c 61 70 70 6c 69 63 61 74  xt/html, applicat
0100  69 6f 6e 2f 78 68 74 6d 6c 2b 78 6d 6c 2c 61 70  ion/xhtml l+xml,ap
0110  70 6c 69 63 61 74 69 6f 6e 2f 78 6d 6c 3b 71 3d  plicatio n/xml;q=
0120  30 2e 39 2c 69 6d 61 67 65 2f 61 76 69 66 2c 69  0.9,imag e/avif,i
0130  6d 61 67 65 2f 77 65 62 70 2c 2a 2f 2a 3b 71 3d  mage/web p,/*;q=
0140  30 2e 38 0d 0a 41 63 63 65 70 74 2d 4c 61 6e 67  0.8..Acc ept-Lang
0150  75 61 67 65 3a 20 64 61 2c 65 6e 2d 55 53 3b 71  uage: da ,en-US;q
0160  3d 30 2e 37 2c 65 6e 3b 71 3d 30 2e 33 0d 0a 41  =0.7,en; q=0.3..A
0170  63 63 65 70 74 2d 45 6e 63 6f 64 69 6e 67 3a 20  ccept-En coding:
0180  67 7a 69 70 2c 20 64 65 66 6c 61 74 65 0d 0a 43  gzip, de flate..C
0190  6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d  onnectio n: keep-
01a0  61 6c 69 76 65 0d 0a 55 70 67 72 61 64 65 2d 49  alive..U pgrade-I
01b0  6e 73 65 63 75 72 65 2d 52 65 71 75 65 73 74 73  nsecure- Requests
01c0  3a 20 31 0d 0a 49 66 2d 4d 6f 64 69 66 69 65 64  : 1..If- Modified
01d0  2d 53 69 6e 63 65 3a 20 54 75 65 2c 20 31 32 20  -Since: Tue, 12
01e0  53 65 70 20 32 30 32 33 20 30 35 3a 35 39 3a 30  Sep 2023 05:59:0
01f0  31 20 47 4d 54 0d 0a 49 66 2d 4e 6f 6e 65 2d 4d  1 GMT..I f-None-M
0200  61 74 63 68 3a 20 22 35 31 2d 36 30 35 32 33 32  atch: "5 1-605232
0210  33 62 36 66 39 33 38 22 0d 0a 0d 0a              3b6f938" ....
  
```

HTTP User-Agent header (http.user_agent), 94 byte(s)

Packets: 193 · Displayed: 6 (3.1%) · Dropped: 0 (0.0%) | Profile: Default

HTTP response message: general format



HTTP response status message

Protocol status line:

- status code
- status phrase

header
lines

data, e.g.,
requested
HTML file

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK

- request succeeded, requested object later in this msg

301 Moved Permanently

- requested object moved, new location specified later in this msg (Location:)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Method types

- GET
 - Retrieves a resource/object
- POST
 - Uploads data (create a resource)
- HEAD
 - Asks server to leave requested object out of response
- PUT
 - uploads file in entity body to path specified in URL field (can replace existing)
- DELETE
 - deletes file specified in the URL field
- CONNECT, OPTIONS, TRACE, PATCH – lesser used

Uploading form input

POST method:

- web page often includes form input
- input is uploaded to server in entity body

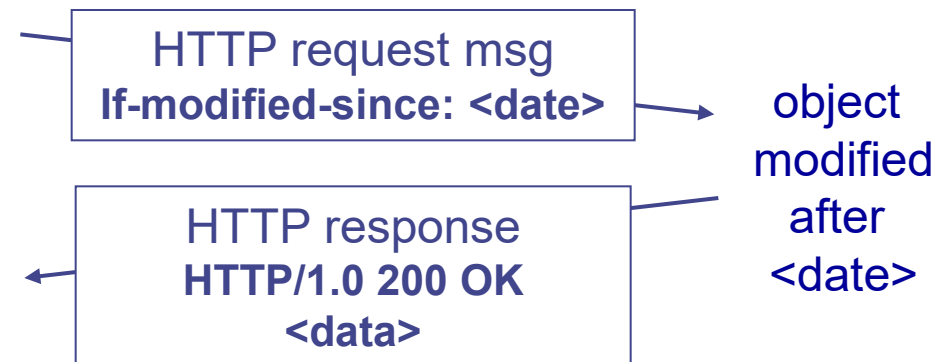
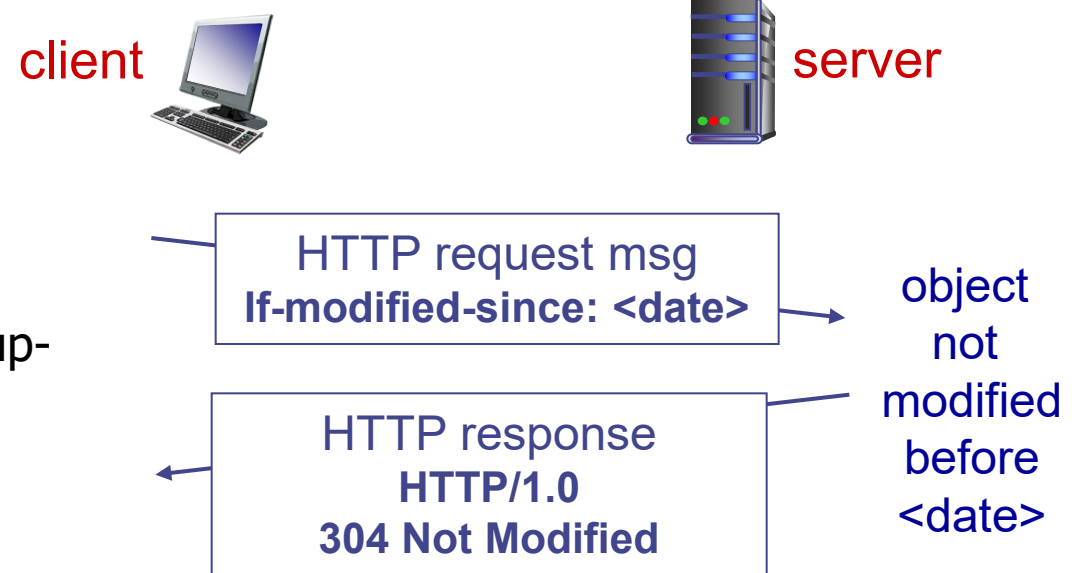
URL method:

- uses GET method
- input is uploaded in URL field of request line: `www.somesite.com/search?room&U181`

Conditional GET

- HTML → small amount of data
- Object → potentially large amounts
- **Goal:** don't send object if **cache** has up-to-date cached version
 - no object transmission delay
 - lower link utilization
 - **Cached resource may be old!**

- **Cache:** specify date of cached copy in HTTP request
`If-modified-since: <date>`
- **Server:** response contains no object if cached copy is up-to-date:
`HTTP/1.0 304 Not Modified`



Talking to hardware

- REST API (Representational State Transfer API)
- A REST API request is composed of four elements: Method, IP, API Version, and Endpoint.



- Example: GET 192.168.1.100/api/v2.0.0/status → get the robot status
- Robot status could be a file (e.g. json, xml) with all relevant status information

A Note on E-mail

- SMTP (Simple Mail Transfer Protocol)
Purpose: Sending emails from a client to a server or between servers.
- POP3 (Post Office Protocol v3)
Purpose: Downloading emails from the server to a local device.
- IMAP (Internet Message Access Protocol)
Purpose: Reading and managing emails stored on a remote server
- Further details are not exam topics

Wireshark Labs

- Lab 2: HTTP
- There are no demands for hand-ins of the Labs, but do the journals described in the labs anyway, for discussions at the class and with your classmates.